

Tema 6

Colecciones



Índice

- Introducción
- Arrays (vectores) y matrices
- Listas
- Pilas
- Colas
- Métodos y propiedades comunes a las colecciones
- Otras colecciones avanzadas



¿Qué son las colecciones?

- Las colecciones son conjuntos organizados de datos de un mismo tipo (int, float, string, object, clases propias...).
- Estos datos se pueden organizar y exponer de diferentes maneras, lo que lleva a que existan diferentes tipos de colecciones.

1. Introducción



- En C#, las **colecciones** son estructuras que permiten almacenar y gestionar grupos de objetos relacionados de manera eficiente.
- Todas las colecciones, excepto los arrays, son **redimensionables** en tiempo de ejecución, por lo que podemos añadir o eliminar objetos durante la ejecución del programa.
- Podemos acceder a los miembros individuales de una colección a través de su índice.

1. Introducción

Colecciones genéricas y no genéricas

- En C#, las colecciones pueden ser de 2 tipos: genéricas o no genéricas.
- Las colecciones **genéricas** definen el tipo de dato que van a contener.
- Las colecciones **no genéricas** no definen ningún tipo de dato, por lo que podemos añadir a ellas cualquier tipo de dato.
- En general, para cada colección existe su versión genérica y su versión no genérica.

1. Introducción

Colecciones genéricas

- Las colecciones **genéricas** ofrecen seguridad de tipos debido a que debemos especificar el tipo de dato que contienen.
- Esto ofrece **seguridad** en tiempo de compilación, evitando posibles errores de ejecución.
- Además, son más **amigables para el programador**, ya que evitamos tener que hacer castings, conversiones y comprobaciones para cada elemento.

1. Introducción

- Las colecciones genéricas también ofrecen un mayor **rendimiento**, tanto en tiempo de ejecución como en consumo de memoria.
- Las colecciones genéricas se definen como `Collection<T>` siendo `T` el tipo de dato que van a contener, por ejemplo:
 - `List<int>`
 - `ArrayList<bool>`
 - `Queue<Car>`
 - `HashSet<Person>`

1. Introducción

Colecciones no genéricas

- Las colecciones **no genéricas** pueden almacenar cualquier tipo de objeto en ellas.
- Esto puede suponer una fuente de **errores en tiempo de ejecución**, ya que podemos no tener controlados todos los casos que nos vamos a encontrar.
- Además, supone un **trabajo extra** al tener que realizar castings y comprobaciones para trabajar con cada elemento.

1. Introducción

- Las colecciones no genéricas ofrecen un **peor rendimiento** debido a que tienen que realizar conversiones internas para cada objeto que almacenan.
- Las colecciones no genéricas se definen como `Collection`, sin tipo asociado:
 - `List`
 - `ArrayList`
 - `Queue`
 - `HashSet`

1. Introducción

- En general, siempre utilizaremos colecciones **genéricas** ya que ofrecen mayor seguridad, rendimiento, facilidad de uso y legibilidad de código.
- El uso de colecciones no genéricas **no es recomendable** y en el desarrollo moderno en C# está reservado únicamente para proyectos que tienen que interactuar con código heredado o sistemas externos que utilizan estas colecciones no genéricas.

1. Introducción

Definición

- Los **arrays o vectores** son una de las colecciones más simples y fundamentales en la programación.
- Un array es una **colección fija e indexada** de elementos del mismo tipo, a los que se accede mediante un índice.
- Los arrays son útiles cuando se conoce de antemano el **número de elementos** que se desean almacenar.

2. Arrays (vectores) y matrices

- Una vez que se define el **tamaño** de un array, no se puede redimensionar, por lo que no se pueden añadir o eliminar elementos.
- Todas las colecciones comienzan con el **índice 0** (primer elemento de la colección) y finalizan en el índice '*tamaño - 1*' (último elemento).
- Intentar acceder a un elemento fuera del rango de una colección dará como resultado una **excepción** en tiempo de ejecución (`IndexOutOfRangeException`).

2. Arrays (vectores) y matrices

Sintaxis básica

- Para declarar un array, debemos utilizar el tipo de datos que va a contener seguido de corchetes (`[]`) y después el nombre de la variable.
- Para inicializar un array, debemos asignar la variable la expresión `new TipoDato[numeroElementos]`.
- Para acceder a un miembro utilizamos la expresión `variable[indice]`.

2. Arrays (vectores) y matrices

Ejemplo

2. Arrays (vectores) y matrices

```
1  int[] numbers = new int[5];
2  // Declaramos e inicializamos un array de 5 números
3
4  int firstElement = numbers[0];
5  // Accedemos al primer elemento del array
6
7  int lastElement = numbers[4];
8  // Accedemos al último elemento del array
9
10 int errorElement = numbers[5];
11 // Error: Índice fuera de rango
```

Asignación de valores

- Aunque hemos dicho que a un array no se le pueden añadir o eliminar elementos, sí que podemos **asignarle valor a los elementos** que contiene.
- Para ello, utilizamos la expresión `variable[indice] = valor;`

Ejemplo:

2. Arrays (vectores) y matrices

```
1 numbers[2] = 87;  
2 // Asignamos 87 al tercer elemento  
3  
4 numbers[0] = -17329;
```

- También podemos **simplificar** la creación y asignación de valores mediante el uso de llaves para introducir los valores directamente en la declaración:
- De esta manera asignamos todos los valores del array a la vez que declaramos la variable. No hay que especificar su longitud (será el número de valores añadidos en la asignación).

2. Arrays (vectores) y matrices

```
1 double[] prices = { 7.00, 10.5, 8.1
```

Recorrer un array

- Podemos recorrer secuencialmente un array utilizando sentencias de iteración, habitualmente `for` o `foreach`.
- Utilizamos la expresión `array.Length`; para obtener el número de elementos que contiene.
- Así pues, podemos recorrer un array con la siguiente estructura:

2. Arrays (vectores) y matrices

- Con este código recorreremos el array y podemos tanto leer y operar con cada miembro individual del array como asignar un valor a cada miembro.

2. Arrays (vectores) y matrices

```
1 for(int i = 0; i < array.Length; i++)
2 {
3     // Código con cada elemento del array
4     // al que accedemos con array[i]
5 }
```

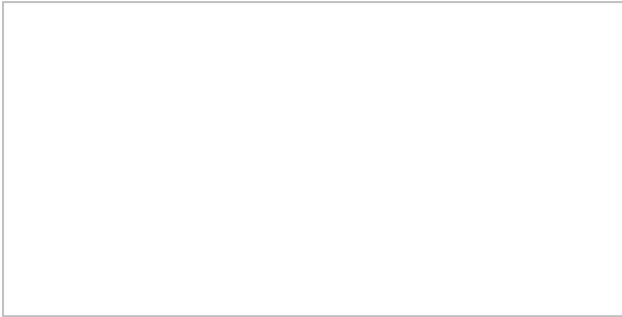
Ejemplo

- En este ejemplo, declaramos un array de nombres (string). Lo recorremos con un primer bucle for asignando un nuevo valor (la cadena en mayúsculas). Posteriormente, un nuevo for imprime por pantalla cada string contenido en el array.

2. Arrays (vectores) y matrices

```
1 string[] names = { "Manuel", "A  
2     "María", "Pedro", "Ana" };  
3  
4 for (int i = 0; i < names.Length  
5 {
```

```
6     names[i] = names[i].ToUpper
7 }
8
9 for (int i = 0; i < names.Length
10 {
11     Console.WriteLine(names[i])
```



- También podemos recorrer un array con el iterador `foreach`, que vimos en el tema 4.
- En ese caso, debemos utilizar la sintaxis:
- En cada iteración, elemento contendrá cada uno de los miembros del array.
- En este caso, **no podemos modificar** el contenido del iterador, únicamente acceder a él.

2. Arrays (vectores) y matrices

```
foreach(tipo_dato elemento in a1  
{  
    // Operaciones con cada eler
```

Ejemplo

- En este ejemplo, en cada iteración la variable `name` toma el valor de cada uno de los miembros del array `names`.
- **No podemos asignarle nuevos valores** ni acceder al índice.

2. Arrays (vectores) y matrices

```
1 foreach (string name in  
2 {  
3     Console.WriteLine(name);  
}
```

Vídeo práctico

2. Arrays (vectores) y matrices

Vídeo práctico - código

2. Arrays (vectores) y matrices

```
1  int[] numbersArray;  
2  numbersArray = new int[10];  
3  
4  int[] numbersArray2 = { 4, -58, 7, 100, -32, 4, 4  
5  
6  numbersArray[0] = 255;  
7  
8  // Console.WriteLine(numbersArray2[3]);  
9  
10 for (int i = 0; i < numbersArray.Length; i++)  
11 {  
12     numbersArray[i] = i * 10;  
13 }  
14  
15 foreach (int number in numbersArray)  
16 {  
17     Console.WriteLine(number);  
18 }  
19  
20 string[] daysOfWeek = new string[7];
```

Arrays multidimensionales (matrices)

- En C# también podemos definir arrays multidimensionales o **matrices**.
- Estas matrices pueden ser de cualquier número de dimensiones, aunque lo habitual es que sean **bidimensionales**.
- Un array bidimensional representa una matriz matemática o una tabla de datos, con filas y columnas.

2. Arrays (vectores) y matrices

- Para inicializar un array multidimensional utilizamos la siguiente expresión:
- Para asignar valores, debemos anidar llaves, por ejemplo:
- También podemos unir declaración y asignación en 1 sentencia única (quitamos los 2 `new int[, ]`).

2. Arrays (vectores) y matrices

```
tipo dato[] nombre = new tipo dato[anch
```

```
1 int[,] matrix = new int[
2
3 matrix = new int[, ]
4 {
```

```
5 {4, 180, -42, 4},  
6 {12, 48, 22, 10},  
7 {132, -21, 8087, 20},  
8 };
```

- Para acceder o asignar un valor se utiliza la expresión `array[filas, columna]`.
- Para saber el tamaño de una matriz multidimensional, podemos utilizar la expresión `array.GetLength(dimension);`
- Esto nos devuelve el número de elementos de la dimensión especificada.

2. Arrays (vectores) y matrices

```
1 int row2col1 = matrix[2, 1];  
2 // Con los datos del ejemplo anterior,  
3 // row3col2 tiene el valor -21  
4  
5 matrix[0, 0] = 27;  
6 // Además, 27 es el primer elemento de 1
```

Ejemplo:

- Con la matriz del ejemplo anterior, podemos utilizar `GetLength(0)` para obtener el número de filas y `GetLength(1)` para obtener el número de columnas.
- Podemos seguir obteniendo la longitud de cada una de las dimensiones si se trata de un array con más de 2 dimensiones.

2. Arrays (vectores) y matrices

```
1 int rows = matrix.GetLength  
2 // rows es 3  
3  
4 int columns = matrix.GetLen
```

- Sabiendo el número de elementos de cada dimensión, podemos **iterar** los elementos de una matriz multidimensional mediante bucles for anidados.
- También podemos utilizar un bucle foreach, en cuyo caso nos devuelve los elementos en orden **secuencial**, primero todos los de la primera fila, a continuación los de la segunda fila, etc.

2. Arrays (vectores) y matrices

Ejemplo:

2. Arrays (vectores) y matrices

```
1 for (int i = 0; i < matrix.GetLength(0)
2 {
3     for (int j = 0; j < matrix.GetLengt
4     {
5         Console.Write(matrix[i, j] + "\
6     }
7     Console.WriteLine();
```

```
1 foreach (int i in matl
2 {
3     Console.Write(i +
4     ,
```



Array de arrays (jagged arrays)

- C# también permite definir arrays cuyo tipo de dato sea otro array.
- Los arrays de arrays (**jagged arrays** en inglés) nos permiten tener filas de diferente número de elementos.
- Esto nos proporciona mayor **flexibilidad** al poder definir estructuras irregulares, pero ello también supone una mayor dificultad a la hora de trabajar.

2. Arrays (vectores) y matrices

- Para inicializar o acceder a un array de arrays utilizamos dobles corchetes:

2. Arrays (vectores) y matrices

```
1  int[][] jaggedArray = new int[3][];
2
3  jaggedArray[0] = [1, 2, 3, 4, 5];
4  jaggedArray[1] = [158, 44];
5  jaggedArray[2] = [-1, 0, 5, 72];
6
7  int[][] jaggedArray2 =
8  [
9      [48, -3, 25, 71, 9],
10     [0, 22, -4, 16],
11     [111, 222]
12 ];
13
14 jaggedArray[0][2] = 88;
15 // Asigna 88 al tercer elemento de la pri
16
17 int element = jaggedArray2[1][3];
18 // element tiene el valor 16
```

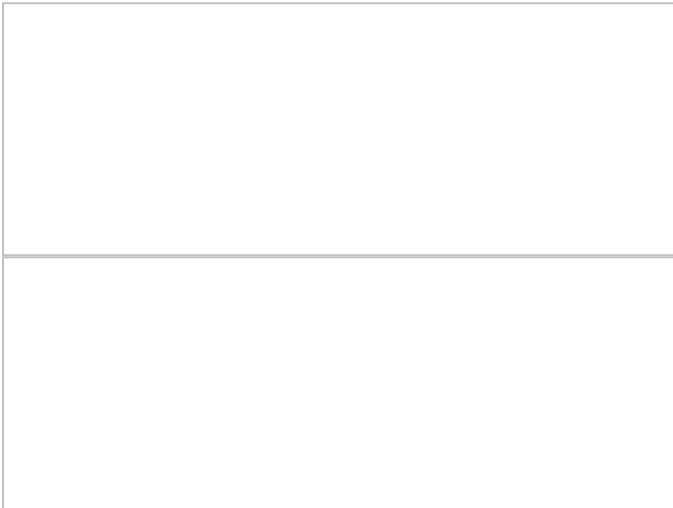
- Podemos iterar con bucles for, teniendo en cuenta la **longitud** de cada array individual en el bucle interno.
- También podemos utilizar un bucle foreach, pero en este caso cada elemento de la iteración es un **array**, por lo que debemos utilizar otro bucle foreach o un bucle for para recorrer todos los elementos.

2. Arrays (vectores) y matrices

Ejemplo

2. Arrays (vectores) y matrices

```
1 for (int i = 0; i < jaggedArray.Length;  
2 {  
3     for (int j = 0; j < jaggedArray[i].Length;  
4     {  
5         Console.Write(jaggedArray[i][j] + " ");  
6     }  
7     Console.WriteLine();  
}
```



```
1 foreach (int[] array in jaggedArray)  
2 {  
3     foreach (int i in array)  
4     {  
5         Console.Write(i + " ");  
6     }  
7     Console.WriteLine();  
}
```

```
5         Console.Write(i + "\n");  
6     }  
7  
8     Console.WriteLine();
```

Vídeo práctico

2. Arrays (vectores) y matrices

Vídeo práctico - código

2. Arrays (vectores) y matrices

```

1  int[,] matrix = new int[5, 2];
2
3  int[,] matrix2 =
4  {
5      { 1, 3, 45 },
6      { 4, 130, -42 },
7      { 42, 42, 1111 },
8      { 10, -1380, 32 }
9  };
10
11 matrix[3, 1] = 4;
12
13 for (int i = 0; i < matrix2.GetLength(0); i++)
14 {
15     for (int j = 0; j < matrix2.GetLength(1); j++)
16     {
17         Console.Write(matrix2[i, j] + "\t");
18     }
19     Console.WriteLine();
20 }
21

```

- En programación, una **lista** es una estructura de datos que organiza y almacena elementos de manera secuencial y ordenada.
- En C#, las listas son estructuras **dinámicas** que, a diferencia de los arrays, pueden cambiar de tamaño de forma automática cuando se añaden o eliminan elementos.
- Para trabajar con listas en C#, utilizamos el tipo de datos **List<T>** (siendo T el tipo de dato de los elementos de la lista).

3. Listas

- Las listas nos ofrecen ventajas a nivel de **flexibilidad** respecto a los arrays.
- Podemos añadir elementos, eliminar, reordenar, buscar, obtener sublistas...
- Esta flexibilidad no es gratuita y puede suponer una eficiencia menor que los arrays, dependiendo del tipo de uso que le vayamos a dar será recomendable utilizar uno u otro tipo de colección.

3. Listas

- Las listas, al igual que todos los tipos de referencia en C#, son `null` en el momento de declararlas.
- Esto significa que debemos inicializarlas antes de poder trabajar con ellas, de lo contrario obtendremos una excepción en tiempo de ejecución.
- Para inicializarlas utilizamos la expresión:
- Siendo `T` el tipo que va a contener la lista.

3. Listas

```
List<T> list = new List<
```

- También podemos declarar y asignar valores en la misma sentencia, como ocurría con los arrays, utilizando llaves ({}).
- Ejemplo:
- Desde la versión 12.0 de C#, también podemos utilizar **expresiones de colección** para asignar valores sin el new, usando corchetes ([]):

3. Listas

```
1 List<string> namesList = new List<string>();  
2  
3  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102  
103  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119  
120  
121  
122  
123  
124  
125  
126  
127  
128  
129  
130  
131  
132  
133  
134  
135  
136  
137  
138  
139  
140  
141  
142  
143  
144  
145  
146  
147  
148  
149  
150  
151  
152  
153  
154  
155  
156  
157  
158  
159  
160  
161  
162  
163  
164  
165  
166  
167  
168  
169  
170  
171  
172  
173  
174  
175  
176  
177  
178  
179  
180  
181  
182  
183  
184  
185  
186  
187  
188  
189  
190  
191  
192  
193  
194  
195  
196  
197  
198  
199  
200  
201  
202  
203  
204  
205  
206  
207  
208  
209  
210  
211  
212  
213  
214  
215  
216  
217  
218  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000
```

Operaciones sobre listas

- C# nos ofrece una serie de mecanismos para trabajar con listas.
- Inserción, borrado, búsqueda, ordenación, conversión... existen múltiples opciones que nos garantizan flexibilidad y facilidad de trabajo.
- A continuación vamos a ver las operaciones más importantes.

3. Listas

- **Add(elemento)**: añade un nuevo elemento al final de la lista.
- Esta operación nos permite añadir elementos sin necesidad de especificar el índice.
- Como vemos, la lista crece de tamaño de manera dinámica, no hay que especificar su capacidad de antemano.

3. Listas

```
1 List<string> namesList = new List<  
2  
3 namesList.Add("Enrique");  
4 namesList.Add("Ana");  
5 namesList.Add("Máximo");
```

- **Insert(indice, elemento)**: inserta un nuevo elemento en la posición indicada.
- Esta operación mueve los elementos posteriores a dicho índice, aumentando la posición de cada uno de ellos en uno.
- En el ejemplo de la lista anterior, acabaría con los valores "Enrique", "Juan", "Ana", "Máximo", "Ana".

3. Listas

```
1 namesList.Insert(1, 'E');
```

- **Remove(elemento)**: elimina de la lista la primera aparición del elemento especificado como parámetro.
- Si un elemento está repetido más de una vez, únicamente elimina la **primera aparición**. Si el elemento no se encuentra en la lista, no hace nada.
- En el caso de la lista original, quedarían los valores "Máximo" y "Ana".

3. Listas

```
1 namesList.Remove("Enri
```

- **RemoveAt(indice)**: elimina de la lista el elemento que se encuentra en el índice especificado.
- En el caso de la lista original, quedarían los valores "Ana", "Máximo" y "Ana".
- Si especificamos un límite fuera del rango de la lista obtendremos un **error en tiempo de ejecución**.

3. Listas

```
1 namesList.Remove
```

- **IndexOf(elemento)**: devuelve un entero con el índice que ocupa en la lista el elemento pasado como parámetro.
- Si existe más de una ocurrencia del elemento, devuelve el índice del **primero** de ellos. También podemos usar **LastIndexOf** para obtener el índice de la última ocurrencia del elemento.
- Si el elemento no está contenido en la lista, ambos métodos devuelven -1.

3. Listas

```
1 int maximoIndex = namesList.IndexOf(
```

- **Sort()**: reordena la lista según el criterio de reordenación del tipo de dato contenido en ella.
- En el caso de los tipos numéricos, se ordena de menor a mayor, en el caso de los `strings` se ordena en orden alfabético. En el caso de tipos personalizados, podremos definir el orden que queramos.
- La lista original ahora contendrá los valores "Ana", "Ana", "Enrique" y "Máximo", en ese orden.

3. Listas

```
1 namesList.Sort()
```

Listas vs arrays

- Como hemos visto, las listas nos ofrecen una mayor flexibilidad respecto a los arrays, pero esta no es "gratuita".
- Las listas ocupan más memoria y el acceso a sus elementos es más lento que en los arrays.
- Dependiendo del uso que necesitemos, vamos a elegir uno u otro tipo de colección.

3. Listas

- Preferimos utilizar **listas** cuando vamos a realizar operaciones de inserción y borrado.
- Preferimos utilizar **arrays** cuando el número de elementos va a ser fijo y vamos a realizar muchas consultas, pero no insertar o borrar elementos.
- Como caso general, **es preferible el uso de listas** a no ser que se cumplan los requisitos concretos para usar arrays.

3. Listas

Vídeo práctico

3. Listas

Vídeo práctico - código

3. Listas

```
1 List<string> stringList = new List<string>() { "string 1"
2
3 stringList.Add("nueva cadena");
4 stringList.Insert(2, "string insertada en 2");
5
6 stringList.Add("...");
7
8 string quixote = "En un lugar de La Mancha";
9
10 stringList.Add(quixote);
11 stringList.Add(quixote);
12
13 bool removedDots = stringList.Remove("...");
14 bool removedAaa = stringList.Remove("aaa");
15
16 int index = stringList.IndexOf("aaa");
17
18 stringList.Sort();
19
20 foreach (string str in stringList)
21 {
```

- Las **pilas** son un tipo de colección que implementan el principio **LIFO** (Last In, First Out), es decir, el último elemento que se añade es el primero en ser retirado.
- Este comportamiento es igual que una pila de papeles o una pila de platos: el último que colocas es el primero que retiras y retirar uno "entre medias" es costoso.
- En C# las pilas son implementadas mediante la colección **Stack<T>**.

4. Pilas

- Las pilas son muy utilizadas en la programación para la resolución de un tipo de problemas concreto que requieren control ordenado de los elementos en un flujo inverso.
- Así pues, se utilizan en **algoritmos recursivos**, en la gestión de excepciones, en el historial de navegación de una aplicación, en algoritmos de búsqueda en profundidad...
- Se trata de una colección **muy eficiente** en cuanto a la inserción, lectura y borrado de elementos en lo alto de la pila.

4. Pilas

- Al igual que con el resto de colecciones, utilizamos un `new` para inicializarlas:
- Si intentamos realizar una operación sobre una pila sin inicializar obtendremos una excepción en tiempo de ejecución.

4. Pilas

```
1 Stack<int> numbersStack = new Stack<>();
```

Operaciones sobre pilas

- Como en el caso de las listas, C# nos ofrece una serie de mecanismos para trabajar con pilas.
- Su uso difiere del de las listas por la propia naturaleza de la colección.
- Vamos a conocer las más importantes.

4. Pilas

- **Push(elemento):** añade un nuevo elemento a la pila. El elemento que se agrega siempre se coloca en la parte superior de la pila.
- Este ejemplo vemos el uso de una pila para definir el flujo de navegación en una aplicación. Cada elemento que añadimos se coloca en la parte superior.

4. Pilas

```
1 Stack<string> navigationHistory = new Stack<string>();
2
3 navigationHistory.Push("Inicio");
4 navigationHistory.Push("Perfil");
```

"Datos personales"
"Perfil"
"Inicio"

- **Pop()**: **devuelve y elimina** el último elemento insertado en la pila (el de más arriba).
- En el ejemplo anterior devolvería y **eliminaría** "Datos personales" de la pila.
- Si hacemos Pop() sobre una lista vacía, se producirá un **error** en tiempo de ejecución.

4. Pilas

```
1 string currentPage = navigationHisto
```

"Perfil"

"Inicio"

- **Peek()**: **devuelve** el último elemento insertado en la pila (el de más arriba) sin eliminarlo.
- En el ejemplo original devolvería "Datos personales" **sin eliminarlo** de la pila.
- Si hacemos Peek() sobre una lista vacía, se producirá un **error** en tiempo de ejecución.

4. Pilas

```
1 string currentPage = navigationHisto
```

"Datos
personales"

"Perfil"
"Inicio"

Uso de pilas

- Aunque podemos acceder a un elemento en concreto utilizando su índice o recorrer mediante un iterador, la potencia de las pilas es su **altísima eficiencia** cuando utilizamos únicamente operaciones Push, Pop y Peek.
- Es útil cuando necesitamos almacenamiento temporal y descartamos los elementos después de consumirlos.
- Para el resto de usos, es preferible utilizar `List<T>`.

4. Pilas

Vídeo práctico

4. Pilas

Vídeo práctico - código

4. Pilas

```
1  static void Main(string[] args)
2  {
3      Stack<string> actionsStack = new Stack<string>();
4
5      DoAction("Abrir documento", actionsStack);
6      DoAction("Escribir texto", actionsStack);
7      DoAction("Poner negrita", actionsStack);
8
9      ShowActionHistory(actionsStack);
10
11     ShowStatus(actionsStack);
12
13     Undo(actionsStack);
14     Undo(actionsStack);
15
16     ShowStatus(actionsStack);
17 }
18
19 public static void DoAction(string action, Stack<string> actionsStack)
20 {
```

- Las **colas** son un tipo de colección que implementan el principio **FIFO** (First In, First Out), es decir, el primer elemento que se añadió es el primero en ser retirado.
- Este comportamiento es análogo a una cola de espera en un mostrador: el primero que llegó es el primero que es atendido.
- En C# las colas son implementadas mediante la colección **Queue<T>**.

5. Colas

- Las colas son **muy utilizadas** en la programación para resolver problemas de espera secuencial.
- Así, encontramos colas, por ejemplo, en una impresora, en buffers de datos, en peticiones a servidores web, en sistemas de llamadas de atención al cliente y un larguísimo etcétera.

5. Colas

Operaciones sobre colas

- Enqueue(elemento): añade un nuevo elemento **al final** de la cola.
- Cada nuevo elemento que se encola se sitúa detrás de los que ya estaban.

5. Colas

```
1 Queue<int> numbersQueue = new Queue<>();  
2  
3 numbersQueue.Enqueue(5);  
4 numbersQueue.Enqueue(-20);  
5
```

5
-20
1573

- Dequeue(): **devuelve y elimina** el primer elemento de la cola (el que se encoló en primer lugar).
- En el ejemplo anterior, Dequeue() devolvería 5 (el primer número que se añadió a la cola) y lo eliminaría.
- Si la cola se encuentra vacía, Dequeue() producirá en una excepción.

5. Colas

```
1 int firstNumber = numbersQueue.De  
2 // firstNumber es 5
```

-20

1573

- `Peek()`: **devuelve** el primer elemento de la cola (el que se encoló en primer lugar) **sin eliminarlo**.
- En el ejemplo original, `Peek()` devolvería 5 (el primer número que se añadió a la cola) pero no lo eliminaría, seguiría siendo el primer elemento.
- Si la cola se encuentra vacía, `Peek()` producirá en una excepción.

5. Colas

```
1 int firstNumber = numbersQueue.Pe  
2 // firstNumber == 5
```

5
-20
1573

Vídeo práctico

5. Colas

Vídeo práctico - código

5. Colas

```
1  static void Main(string[] args)
2  {
3      Queue<string> clientsQueue = new Queue<string>();
4
5      ResponseCall(clientsQueue);
6
7      AddClient("Juan Alfaro", clientsQueue);
8      AddClient("Ana Álvarez", clientsQueue);
9      AddClient("Antonio Robles", clientsQueue);
10
11     ShowStatus(clientsQueue);
12     ListClients(clientsQueue);
13
14     ResponseCall(clientsQueue);
15     ShowStatus(clientsQueue);
16     ListClients(clientsQueue);
17 }
18
19 private static void AddClient(string name, Queue<string>
```

- Además de las **operaciones** específicas de cada tipo de colección que hemos visto, existen una serie de herramientas comunes a las colecciones en C#.
- Estos métodos incluyen diferentes búsquedas, conversiones, limpiezas...
- Todos los métodos que vamos a ver son aplicables a `List<T>`, `Stack<T>`, `Queue<T>` y otras colecciones, pero **no todos son aplicables a los arrays**.

6. Métodos y propiedades comunes a las colecciones

Count

- La propiedad **Count** sobre una colección nos devuelve el número de elementos que contiene.
- Es el equivalente a `Length` en arrays (que no tienen propiedad `Count`).
- Ejemplo:

6. Métodos y propiedades comunes a las colecciones

```
1 int namesNumber = namesList.Count;
```

Contains

- El método **Contains(elemento)** devuelve un bool que indica si el elemento pasado como parámetro está contenido en una colección.
- Ejemplo:

6. Métodos y propiedades comunes a las colecciones

```
1 bool isInicio = navigationHistory.Contains("Inicio")
2 // isInicio es true en el ejemplo de las pilas
3
4 bool isSettings = navigationHistory.Contains("Confi
5 // isSettings es false en el ejemplo de las pilas
```

ElementAt

- El método **ElementAt(indice)** devuelve el elemento que se encuentra en la posición 'indice'.
- Es eficiente con listas y arrays, pero no con pilas y colas.
- Ejemplo:

6. Métodos y propiedades comunes a las colecciones

```
1 int number1 = numbersQueue.ElementAt(1)  
2 // number1 es 20 en el ejemplo de los
```

Clear

- El método **clear()** elimina todos los elementos de una colección.
- Este método no destruye la instancia (es decir, no la hace `null`), únicamente borra todos su contenido, manteniendo la colección.
- **NO SE PUEDE UTILIZAR EN ARRAYS.**
- Ejemplo:

6. Métodos y propiedades comunes a las colecciones

```
1 namesList.Clear();  
2 // namesList no tiene ningún por
```

ToArray

- El método **ToArray()** devuelve un array de tipo T (tipo de datos de la colección) con todos los miembros de la colección.
- Cada colección ordena los elementos según su método de extracción (es decir, del primero al último en listas y colas y del último al primero en pilas).
- Ejemplo:

6. Métodos y propiedades comunes a las colecciones

```
1 string[] historyArray = navigationHistory.ToArray();
```

LINQ

- Además de estos métodos, las colecciones en C# soportan consultas LINQ sobre sus miembros.
- Esto nos proporciona un marco sobre el que realizar consultas y **operaciones complejas** que nos ofrecen funcionalidad avanzada, desde búsquedas en la colección a agregaciones o iteradores avanzados.
- En el tema 11 veremos LINQ en detalle.

6. Métodos y propiedades comunes a las colecciones

Vídeo práctico

6. Métodos y propiedades comunes a las colecciones

Vídeo práctico - código

6. Métodos y propiedades comunes a las colecciones

```
1  int[] intArray = { 381, 280, -21, 1, 0, 0, 42, -8 };
2  List<double> doubleList = [8.45, -666, 0, 0, .448, 5898,
3  Stack<string> stringStack = new Stack<string>();
4  stringStack.Push("asdf");
5  stringStack.Push("ñlkj");
6  stringStack.Push("hola mundo");
7  stringStack.Push("Curso C# 13");
8  Queue<char> charQueue = new Queue<char>();
9  charQueue.Enqueue('a');
10 charQueue.Enqueue('b');
11 charQueue.Enqueue('#');
12 charQueue.Enqueue('@');
13 charQueue.Enqueue('j');
14
15 Console.WriteLine("Número de elementos de intArray: " + i
16 Console.WriteLine("Número de elementos de doubleList: " +
17 Console.WriteLine("Número de elementos de stringStack: "
18 Console.WriteLine("Número de elementos de charQueue: " +
19
20 Console.WriteLine();
21
```

- Además de las colecciones que hemos visto, C# implementa una gran cantidad de colecciones de carácter avanzado.
- Estas colecciones nos permiten trabajar con requisitos específicos, desde acceso concurrente a colecciones para trabajar con conjuntos algebraicos.
- Vamos a conocer alguna de estas colecciones sin entrar en detalle, pues queda fuera del alcance del curso.

7. Otras colecciones avanzadas

Dictionary<TKey, TValue>

- En C#, un **diccionario** es una colección que almacena pares de clave-valor, donde cada clave (de tipo TKey) es única y está asociada a un valor (de tipo TValue) específico.
- Permite buscar por clave o por valor y son muy eficientes en búsquedas, inserciones y eliminaciones.

7. Otras colecciones avanzadas

HashSet<T>

- Es una colección que almacena elementos únicos, es decir, **no permite duplicados**.
- Utiliza una función hash para organizar los datos internamente, lo que permite un acceso muy rápido a las operaciones de inserción, eliminación y búsqueda.
- Es ideal para trabajar con **grandes conjuntos** de datos cuando estos son únicos.

7. Otras colecciones avanzadas

ConcurrentQueue<T>

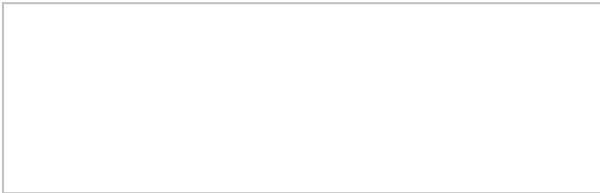
- Es una colección segura para hilos que implementa una cola siguiendo el principio **FIFO** (First In, First Out), diseñada para entornos **concurrentes** donde varios hilos pueden acceder y modificar la cola simultáneamente.
- Utiliza una función hash para organizar los datos internamente, lo que permite un acceso muy rápido a las operaciones de inserción, eliminación y búsqueda.
- Es ideal para escenarios multihilo o concurrentes.

7. Otras colecciones avanzadas

PriorityQueue<TElement, TPriority>

- Es una colección que permite almacenar elementos asociados con una **prioridad** y procesarlos en función de esta prioridad (los elementos con mayor prioridad se atienden antes).
- La cola usa una estructura interna eficiente para garantizar que las operaciones de inserción y extracción mantengan la prioridad correcta.
- Es útil para problemas que requieren procesamiento en orden de prioridad.

7. Otras colecciones avanzadas



En este tema hemos conocido las principales colecciones de C#, que necesitaremos para almacenar los datos de nuestros programas. También hemos aprendido distintos métodos para trabajar con ellas y cuándo utilizar una u otra en función de nuestros requisitos.

Speaker notes